



UNIVERSIDAD DE GRANADA

DESARROLLO SOFTWARE

PRACTICA 1

**Castro Reyes, Adrian
Romero Orantes-Zurita, Alberto
Gea Martínez, Alejandro**

FECHA: 03/2026

Enlace a repositorio en GitHub

Índice de contenido

Índice	2
1. Ejercicio 1	3
1.1. Diagrama UML	3
1.2. Patrón utilizado	3
1.3. Cambios realizados	4
1.4. Ejecución	4
2. Ejercicio 2	6
2.1. Diagrama UML	6
2.2. Patrón utilizado	7
2.3. Cambios realizados	7
2.4. Ejecución	7
3. Ejercicio 3	8
3.1. Diagrama UML	8
3.2. Patrón utilizado	8
3.2.1. Estrategias implementadas	8
3.3. Cambios realizados	8
3.4. Ejecución	9
4. Ejercicio 4	11
4.1. Diagrama UML	11
4.2. Patrón utilizado	11
4.3. Cambios realizados	12
4.4. Ejecución	12

1. Ejercicio 1

1.1.- Diagrama UML

El siguiente diagrama UML muestra la estructura de clases utilizada en el programa. En él se representan las clases abstractas Partida y Jugador, así como sus especializaciones PartidaCompetitiva, PartidaCasual, JugadorCompetitivo y JugadorCasual. También se incluye la interfaz FactoriaPartidayJugador y sus implementaciones FactoriaCompetitiva y FactoriaCasual.

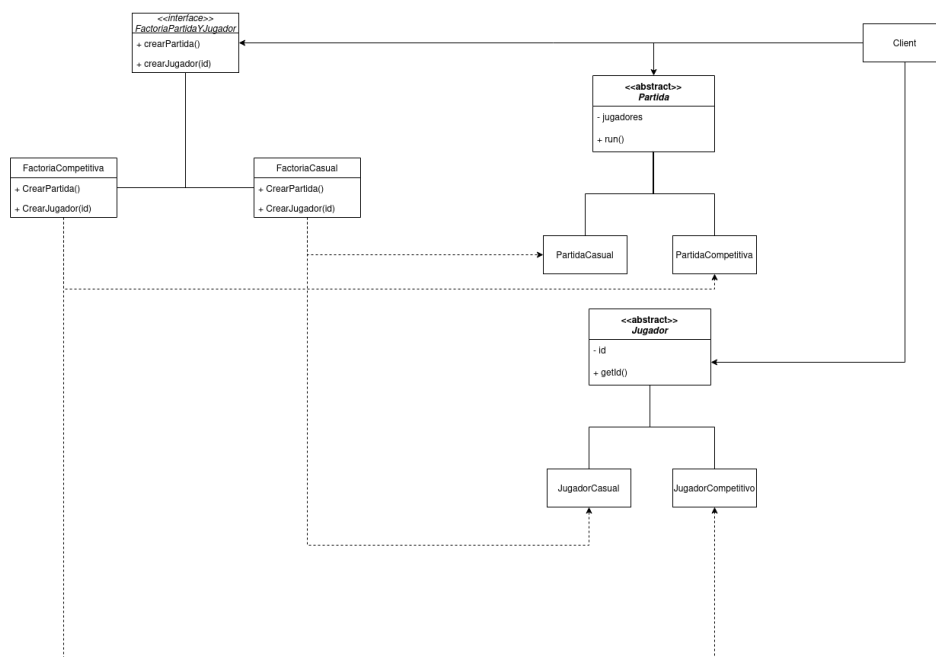


Figura 1.1: Diagrama de clases UML

1.2.- Patrón utilizado

Para la implementación del ejercicio se ha utilizado el patrón Factoría Abstracta. Este patrón permite crear familias de objetos relacionados sin especificar sus clases concretas.

En este caso, la interfaz FactoriaPartidayJugador define los métodos necesarios para crear partidas y jugadores. Posteriormente, las clases FactoriaCompetitiva y FactoriaCasual implementan dicha interfaz para generar objetos específicos de tipo competitivo o casual.

De esta forma, se garantiza que cada tipo de partida cree los jugadores adecuados sin acoplar el código a clases concretas, facilitando la escalabilidad y el mantenimiento del sis-

tema.

1.3.- Cambios realizados

Se ha diseñado una estructura basada en clases abstractas e interfaces para permitir la creación de distintos tipos de partidas y jugadores manteniendo una organización clara del código.

La clase abstracta Partida implementa la interfaz Runnable para permitir la ejecución simultánea de múltiples partidas mediante hebras. Además, se ha utilizado una colección de tipo ArrayList para almacenar los jugadores de cada partida.

Las clases concretas PartidaCompetitiva y PartidaCasual implementan el comportamiento específico de cada tipo de partida, incluyendo la simulación de abandonos simultáneos de jugadores.

1.4.- Ejecución

El programa solicita al usuario el número de jugadores que participarán en cada partida. A continuación, se crean dos partidas distintas (competitiva y casual) utilizando sus respectivas factorías.

Posteriormente, se añaden los jugadores a cada partida y se crean dos hebras que permiten ejecutar ambas partidas de forma simultánea. Cada partida tiene una duración total de 60 segundos, dividida en dos intervalos de 30 segundos separados por la simulación de abandonos de jugadores.

Finalmente, se muestra por pantalla el desarrollo de cada partida y los abandonos producidos.

```
Introduce el número inicial de jugadores (N): 20

--- INICIANDO SIMULACIÓN DE 60 SEGUNDOS ---
[ PartidaCompetitiva ] Iniciada con: 20 jugadores
[ PartidaCasual ] Iniciada con: 20 jugadores
[Partida competitiva] 4 jugadores han abandonado la partida
[Partida casual] 2 jugadores han abandonado la partida
[ PartidaCompetitiva ] Finalizada. Jugadores restantes 16
[ PartidaCasual ] Finalizada. Jugadores restantes 18
--- AMBAS PARTIDAS HAN TERMINADO CORRECTAMENTE ---
```

Figura 1.2: Ejemplo de ejecución del programa

2. Ejercicio 2

Desarrolla un programa en Python que utilice la API de Hugging Face para interactuar con modelos de lenguaje (LLM) y procesar texto.

2.1.- Diagrama UML

El siguiente diagrama UML representa la estructura de clases utilizada en el programa. En él se observa la clase abstracta LLM que define la operación principal `generate_summary()`. La clase `BasicLLM` implementa esta funcionalidad básica conectándose a la API de Hugging Face para generar resúmenes de texto.

Además, se utiliza una jerarquía de clases basada en Decorator, donde la clase abstracta `Decorator` mantiene una referencia a un objeto de tipo LLM. A partir de ella se derivan `TranslationDecorator` y `SentimentDecorator`, que añaden funcionalidades adicionales como la traducción del resumen y el análisis de sentimiento.

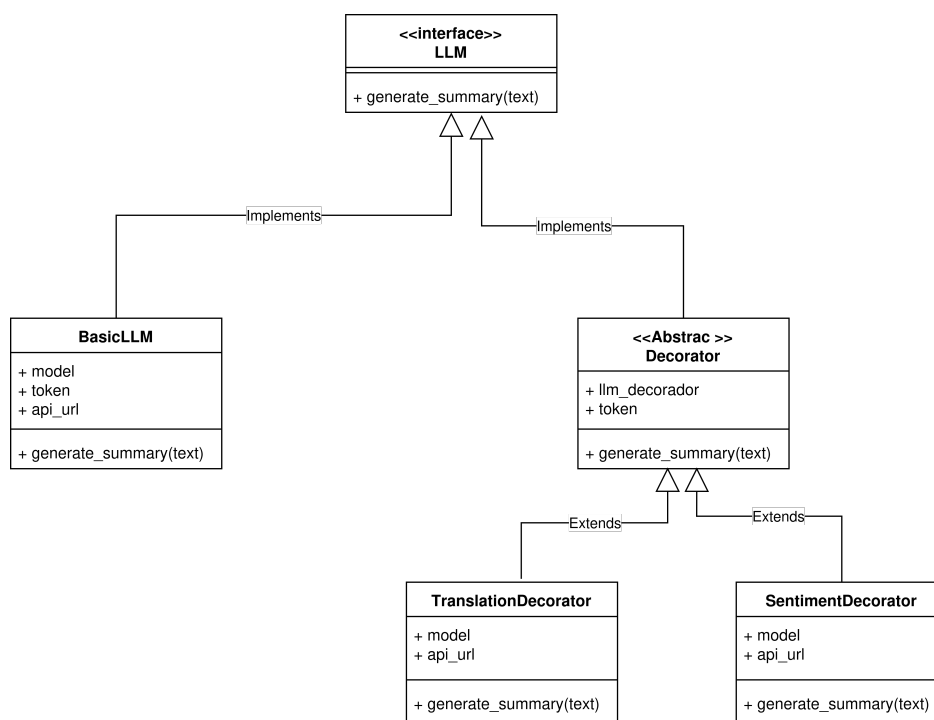


Figura 2.1: Diagrama de clases UML

2.2.- Patrón utilizado

Para la implementación de este ejercicio se ha utilizado el patrón Decorador para añadir nuevas funcionalidades al BasicLLM, como pueden ser la traducción o el análisis de sentimiento, de manera dinámica, envolviendo el objeto original mediante clases decoradoras.

2.3.- Cambios realizados

Se ha adaptado el patrón Decorador para trabajar con servicios externos, concretamente la API de Hugging Face. Cada clase decoradora realiza una petición HTTP al modelo correspondiente utilizando la librería requests y procesa la respuesta obtenida en formato JSON.

La clase TranslationDecorator utiliza un modelo de traducción para convertir el texto generado a otro idioma, mientras que la clase SentimentDecorator emplea un modelo de análisis de sentimiento para determinar la polaridad del texto.

La configuración de los modelos y el token de acceso a la API se realiza mediante un archivo externo info.json, lo que permite modificar fácilmente los modelos utilizados sin cambiar el código fuente.

2.4.- Ejecución

Aquí se muestra una captura de la ejecución del programa, en la cual se le pasa un texto en inglés y se obtiene como salida el resumen básico, el resumen traducido, el análisis de sentimiento y la combinación de todos ellos.

```
adrian@adrian-VirtualBox: ~/Documents/GitHub/DS/Prácticas_1/Ejercicio_2$ python3 decorator.py
RESUMEN BASICO
[{'summary text': 'Brawl Stars is a popular multiplayer mobile game developed by Supercell. Players choose from many different characters called brawlers, each with unique abilities and playstyles. The game is known for its fast-paced matches, colorful graphics, and constant updates that introduce new brawler, skins, and events.'}]

RESUMEN TRADUCIDO
[{'summary text': 'Brawl Stars es un popular juego móvil multijugador desarrollado por Supercell. Los jugadores eligen entre muchos personajes diferentes llamados brawlers, cada uno con habilidades y estilos de juego únicos. El juego es conocido por sus partidas rápidas, gráficos coloridos y actualizaciones constantes que introducen nuevos brawler, pieles y eventos.'}]

RESUMEN SENTIMIENTO
4 stars

RESUMEN COMBINADO
4 stars
adrian@adrian-VirtualBox: ~/Documents/GitHub/DS/Prácticas_1/Ejercicio_2$
```

Figura 2.2: Captura ejecución del programa

3. Ejercicio 3

Este ejercicio consiste en el desarrollo de un sistema de extracción de datos (web scraping) orientado a objetos. Se ha tomado como fuente el sitio web Scrape This Site para recolectar estadísticas de equipos de hockey sobre hielo.

3.1.- Diagrama UML

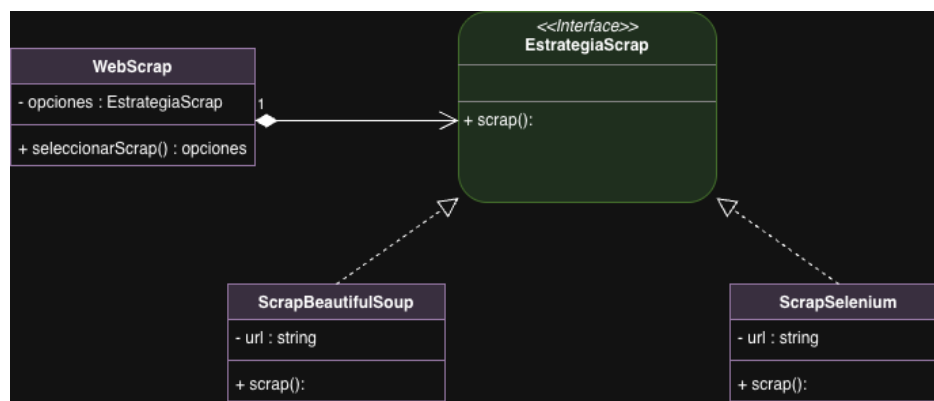


Figura 3.1: Diagrama de clases UML

3.2.- Patrón utilizado

Siguiendo los requisitos de la práctica, se ha implementado el patrón de diseño Strategy. Este patrón estructural permite definir una familia de algoritmos (estrategias de scraping), encapsular cada uno de ellos y hacerlos intercambiables en tiempo de ejecución.

3.2.1.- Estrategias implementadas

- **BeautifulSoupStrategy:** Utiliza la librería BeautifulSoup para procesar el HTML estático obtenido mediante peticiones requests.
- **SeleniumStrategy:** Utiliza Selenium WebDriver para navegar y extraer datos del DOM, permitiendo el procesamiento de contenido dinámico si fuera necesario.

3.3.- Cambios realizados

Se ha diseñado una estructura orientada a objetos basada en una interfaz común para las distintas estrategias de scraping.

Cada estrategia implementa el método scrap(url), encargándose de extraer los datos de la página web indicada. El sistema permite recorrer varias páginas del sitio web y obtener información relevante de los equipos de hockey, incluyendo nombre, año, victorias, derrotas, porcentaje de victorias y estadísticas de goles.

Finalmente, los datos obtenidos se almacenan en un archivo CSV para su posterior análisis.

3.4.- Ejecución

En la ejecución del programa, el usuario selecciona la estrategia de scraping que desea utilizar. A continuación, el sistema recorre las primeras cinco páginas del sitio web y extrae la información de los equipos.

Dependiendo de la estrategia elegida, el scraping se realiza mediante BeautifulSoup o Selenium. Una vez finalizado el proceso, los datos obtenidos se almacenan en un archivo CSV y se muestra un mensaje indicando que el proceso se ha completado correctamente.

	A	B	C	D	E	F	G	H	I
1	Nombre del Equipo	Año	Victorias	Derrotas	Derrotas en tiempo e	Porcentaje de victori	Goles a favor	Goles en contra	Diferencia de goles
2	Boston Bruins	1990	44	24		0.55	299	264	35
3	Buffalo Sabres	1990	31	30		0.388	292	278	14
4	Calgary Flames	1990	46	26		0.575	344	263	81
5	Chicago Blackhawks	1990	49	23		0.613	284	211	73
6	Detroit Red Wings	1990	34	38		0.425	273	298	-25
7	Edmonton Oilers	1990	37	37		0.463	272	272	0
8	Hartford Whalers	1990	31	38		0.388	238	276	-38
9	Los Angeles Kings	1990	46	24		0.575	340	254	86
10	Minnesota North Stars	1990	27	39		0.338	256	266	-10
11	Montreal Canadiens	1990	39	30		0.487	273	249	24
12	New Jersey Devils	1990	32	33		0.4	272	264	8
13	New York Islanders	1990	25	45		0.312	223	290	-67
14	New York Rangers	1990	36	31		0.45	297	265	32
15	Philadelphia Flyers	1990	33	37		0.412	252	267	-15
16	Pittsburgh Penguins	1990	41	33		0.512	342	305	37
17	Quebec Nordiques	1990	16	50		0.2	236	354	-118
18	St. Louis Blues	1990	47	22		0.588	310	250	60
19	Toronto Maple Leafs	1990	23	46		0.287	241	318	-77
20	Vancouver Canucks	1990	28	43		0.35	243	315	-72

Figura 3.2: Captura de ejecución

	A	B	C	D	E	F	G	H	I
100	Montreal Canadiens	1994	18	23		0.375	125	148	-23
107	New Jersey Devils	1994	22	18		0.458	136	121	15
108	New York Islanders	1994	15	28		0.312	126	158	-32
109	New York Rangers	1994	22	23		0.458	139	134	5
110	Ottawa Senators	1994	9	34		0.188	117	174	-57
111	Philadelphia Flyers	1994	28	16		0.583	150	132	18
112	Pittsburgh Penguins	1994	29	16		0.604	181	158	23
113	Quebec Nordiques	1994	30	13		0.625	185	134	51
114	San Jose Sharks	1994	19	25		0.396	129	161	-32
115	St. Louis Blues	1994	28	15		0.583	178	135	4
116	Tampa Bay Lightning	1994	17	28		0.354	120	144	-24
117	Toronto Maple Leafs	1994	21	19		0.438	135	146	-11
118	Vancouver Canucks	1994	18	18		0.375	153	148	5
119	Washington Capitals	1994	22	18		0.458	136	120	16
120	Winnipeg Jets	1994	16	25		0.333	157	177	-20
121	Mighty Ducks of Anaheim	1995	35	39		0.427	234	247	-13
122	Boston Bruins	1995	40	31		0.488	282	269	13
123	Buffalo Sabres	1995	33	42		0.402	247	262	-15
124	Calgary Flames	1995	34	37		0.415	241	240	1
125	Chicago Blackhawks	1995	40	28		0.488	273	220	53
126	Colorado Avalanche	1995	47	25		0.573	326	240	86

Figura 3.3: Captura de ejecución

4. Ejercicio 4

El objetivo es implementar un sistema de validación de credenciales (correo y contraseña) para un servicio de autenticación. Para cumplir con el requisito de añadir validaciones sin modificar las clases existentes, se ha aplicado el patrón de diseño Filtros de Intercepción (Intercepting Filter) utilizando el lenguaje Ruby.

4.1.- Diagrama UML

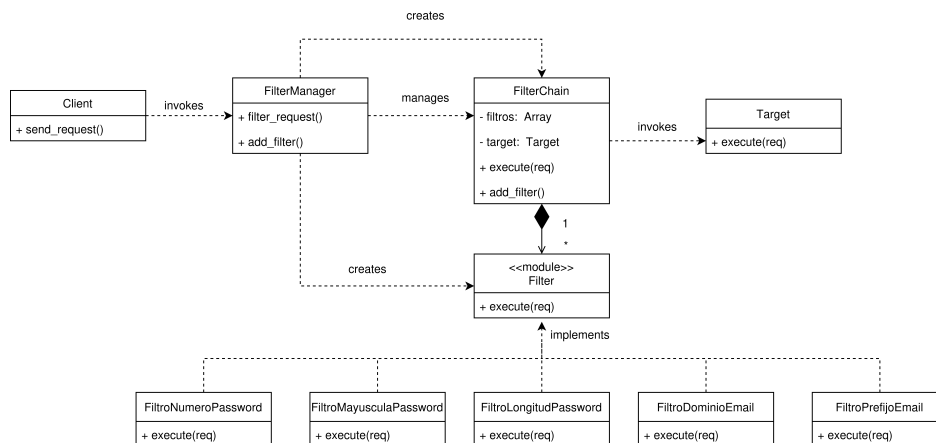


Figura 4.1: Diagrama de clases UML

4.2.- Patrón utilizado

Para la implementación del ejercicio se ha utilizado el patrón Intercepting Filter. Este patrón permite definir una cadena de filtros que procesan una petición antes de que llegue al objeto final encargado de gestionarla.

Cada filtro implementa una única validación, permitiendo separar responsabilidades y mejorar la organización del código. Si alguno de los filtros detecta que la petición no cumple los requisitos establecidos, la cadena de ejecución se interrumpe y la petición es rechazada.

Este enfoque permite añadir nuevas validaciones sin modificar el código existente, facilitando la extensibilidad del sistema.

4.3.- Cambios realizados

Se ha implementado una estructura modular donde cada validación se encuentra en una clase independiente. Para simular el comportamiento de una interfaz en Ruby, se ha utilizado un módulo que define el método `execute(request)`, el cual debe ser implementado por cada filtro.

Se han desarrollado distintos filtros encargados de validar el formato del correo electrónico y los requisitos de la contraseña, como longitud mínima, presencia de mayúsculas y presencia de números.

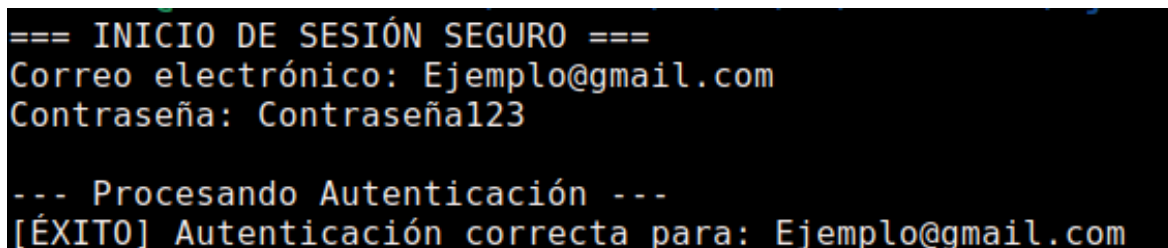
La cadena de filtros se encarga de ejecutar las validaciones en orden, deteniendo el proceso en caso de error.

4.4.- Ejecución

Durante la ejecución del programa, el usuario introduce sus credenciales por consola. La petición pasa a través de la cadena de filtros, donde cada filtro valida una condición específica.

Si todas las validaciones se cumplen, la petición llega al Target final y la autenticación se considera correcta. En caso contrario, el filtro correspondiente intercepta la petición y muestra un mensaje indicando el motivo del error.

Las pruebas realizadas muestran distintos escenarios de ejecución, incluyendo casos en los que la autenticación es aceptada y casos en los que la petición es rechazada por no cumplir los requisitos establecidos.



```
=== INICIO DE SESIÓN SEGURO ===  
Correo electrónico: Ejemplo@gmail.com  
Contraseña: Contraseña123  
  
--- Procesando Autenticación ---  
[ÉXITO] Autenticación correcta para: Ejemplo@gmail.com
```

Figura 4.2: Captura ejecución del programa

```
=== INICIO DE SESIÓN SEGURO ===  
Correo electrónico: @gmail.com  
Contraseña: Contraseña123  
  
--- Procesando Autenticación ---  
Error: Debe contener texto antes de @
```

Figura 4.3: Captura ejecución del programa

```
=== INICIO DE SESIÓN SEGURO ===  
Correo electrónico: Ejemplo@hotmail.com  
Contraseña: Contraseña  
  
--- Procesando Autenticación ---  
[Contraseña] Error: Debe contener al menos un número.
```

Figura 4.4: Captura ejecución del programa

```
=== INICIO DE SESIÓN SEGURO ===  
Correo electrónico: Ejemplo@gmail.com  
Contraseña: DS2026  
  
--- Procesando Autenticación ---  
[Contraseña] Error: Debe tener al menos 8 caracteres
```

Figura 4.5: Captura ejecución del programa